

ARCHITECTURE REVIEW · HEALTHCARE SYSTEMS · EXPERT ANALYSIS

VistA-Backed *Healthcare* Application Architecture

Command-driven, context-aware layered design with modular client support · Expert findings & recommendations

SUMMARY VERDICT		
CONCERN	ASSESSMENT	STATUS
RPC Gateway Layer	Sound. Prioritize error normalization & session management.	SOLID
Execution Context API	Excellent pattern. Invest in declarative validation and context resolution.	EXCELLENT
Command-Driven GUI + Console	Fully achievable. Commands must be self-describing; navigation must be a command.	ACHIEVABLE
Modular Client	Compatible with command design if modules contribute to a shared registry.	COMPATIBLE
Context Schema Governance	Primary architectural risk. Must be designed formally from day one.	RISK

RPC Gateway

Wrapping VistA RPCs behind an HTTP/JSON gateway is the **standard modernization approach** for VistA systems and the correct first architectural boundary. The gateway acts as a thin translator between modern HTTP semantics and the M broker protocol.

KEY RECOMMENDATIONS

- Stateless gateway design

— Keep session and DUZ context in the JSON envelope, not in gateway server state. VistA RPCs are session-bound via M globals; the gateway must manage broker connections per-session with care.
- Versioned RPC mapping registry

— Maintain an explicit registry mapping

rpc_name → {

params,

return_schema }

. Critical when VistA patches change RPC signatures over time.
- Unified error normalization

— VistA error returns are inconsistent across M errors, FileMan errors, and RPC-level errors. Normalize all into a structured error envelope at the gateway — not in the client.
- Authentication boundary

— KACC / CAPRI / CCOW context belongs at this layer, not propagated into client code.

Session handling is non-trivial. M globals are inherently session-scoped. Concurrent request management across multiple clients requires explicit connection pooling or per-request broker instantiation with careful cleanup.

Execution Context API

This is the **strongest and most important design decision** in the architecture. The pattern described is a *Context Object* combined with *Command* and *declarative function metadata* — a well-proven combination for evolving, complex domain models.

Execution Context – Conceptual Structure

```

ExecutionContext = {
  PatientRecord: { dfn, name, dob, ... },
  Visit:         { ien, date, status, ... },
  Provider:      { duz, name, role, ... }
}

APIFunction = {
  name:          "createOrder",
  requires: ["PatientRecord.dfn", "Visit.ien", "Provider.duz"],
  rpc:           "ORWDX SAVE",
  param_map: { ... }
}

// API layer extracts, validates, builds JSON request → RPC Gateway

```

STRENGTHS

- Client is **fully decoupled** from Vista internals — the API layer is a true anti-corruption layer.
- Domain model changes do **not propagate** to client code.
- Same API surface works over any transport: HTTP, named pipe, or in-process call.

CRITICAL DESIGN CONSIDERATIONS

- **Declarative and exhaustive** — check not just object presence but property-level validity: DFN is not null, Visit is not closed, Provider has appropriate role.
context validation
- **Context** — separate from validation. A Patient object may exist in context with an unresolved **resolution step** DFN. Resolution must happen before validation.
- **Context mutation strategy** — decide whether executing a function updates the context (e.g., a created order is added). Mutable context is powerful but harder to debug. Consider an explicit result/event pipeline that proposes context changes the caller must apply.
- **Async and long-running RPCs** — some Vista RPCs are slow. JSON requests should carry a correlation ID to support polling or server-side callbacks.

LAYER 03

Command-Driven Control — GUI & Console from One Model

This is **entirely possible and is the correct goal**. It is the Command pattern taken to its logical conclusion: if every user action maps to a named Command with a declared execution context, the UI is nothing more than a *command dispatcher* — and that dispatcher can be a button, a menu item, a keyboard shortcut, a CLI verb, or a script.

Core Insight: Commands that are self-describing (they declare what context they need) allow the GUI to automatically enable or disable controls based on whether the current context satisfies requirements — eliminating hand-coded enable logic per screen.

Command Registry — Dispatcher Pattern

```
CommandRegistry
└─ Command: { name, requires[], execute(context) → result }

GUI Client
└─ Button "Sign Order" → dispatch("signOrder", currentContext)

ChUI / Console Client
└─ Input "> sign-order" → dispatch("signOrder", currentContext)

Script / Automation
└─ JSON batch          → dispatch sequence against shared context
```

WHAT MAKES THIS WORK

- Commands are self-describing — context-driven enable/disable logic is automatic, not hand-coded per screen.
- A console client can enumerate available commands filtered by current context, giving a natural CLI user experience.
- Testing is deterministic — inject a context, fire a command, assert the result. No UI interaction required.

DISCIPLINE REQUIRED

- All state changes must go through commands — **no back-door direct model mutations** in UI code.
- **Navigation must itself be a command.** Switching screens or views cannot live outside the command system, or the console client loses parity with the GUI.

LAYER 04

Modular Client Architecture

The modular client design does **not contradict** the command-driven model — it *complements* it. The potential conflict arises only if modules maintain isolated command spaces. The solution is for each module to contribute commands to a **shared, global Command Registry** owned by the main application.



RULES TO AVOID OVERCOMPLICATION

- **Modules contribute to the registry; they do not own it.** The main application owns the registry and context — modules are contributors only.
- **Modules are activated, not instantiated per-use.** Load lazily at startup, but do not create and destroy modules per screen transition.
- **Inter-module communication goes through context and commands** Module A needing data from Module B means Module B's command puts that data in shared context. No direct module-to-

only.

module calls.

-
- **Schema evolution is isolated.** When the object model changes, only the affected module and its API mappings change. Other modules remain unaffected as long as context contracts are honored.

Modularity and the command model are natural allies. Modules are the unit of *feature delivery*; commands are the unit of *user interaction*. Keeping these concerns cleanly separated is what prevents complexity from compounding.

⚠ PRIMARY ARCHITECTURAL RISK

The **execution context object graph will grow complex** as the healthcare model evolves. The most common failure mode for this pattern is the context becoming an untyped property bag that different modules interpret inconsistently.

Design a formal **Context Schema** from the start — even before the domain model is finalized. The *shape* of how objects enter, mutate, and leave context must be governed centrally. Without this, diverging interpretations across modules will accumulate as technical debt that is extremely costly to reverse in a healthcare context where data integrity is a regulatory requirement.

CLOSING

Architectural Fit — Overall

The proposed architecture exhibits **strong alignment** between its stated goals and its design choices. The layering — VistA/FileMan → RPC Gateway → Context API → Command Registry → Modular UI Shell — provides genuine separation of concerns and a credible path for incremental expansion of both the domain model and the feature set.

The decision to isolate clients from RPC mechanics via an execution context API is particularly well-suited to a system where the underlying persistence layer (VistA/Mumps) is both highly capable and highly idiosyncratic. It gives future developers a clean surface to work against without requiring VistA expertise at the UI layer.

Consistent governance of the **Context Schema** and the **Command Registry** is where the architecture will either hold together elegantly or fracture under the pressure of feature expansion. Invest in those two structures early.

Healthcare Application Architecture · Expert Analysis

VistA / Mumps / FileMan · Command-Driven Design